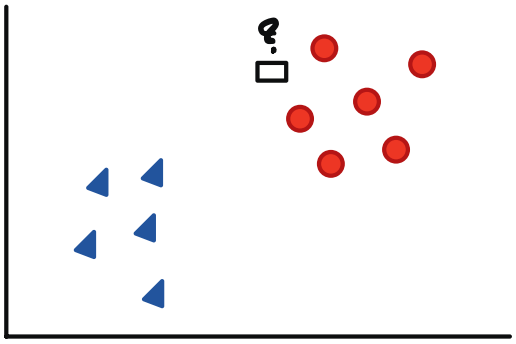
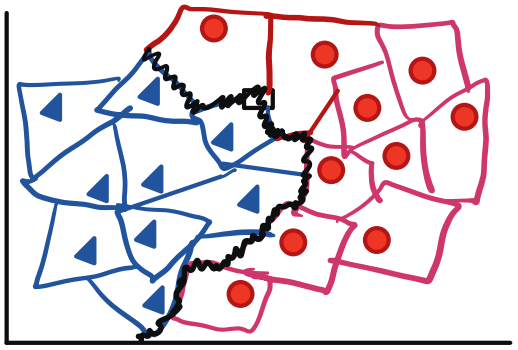
Intuition For KNN



nearby points are red — use this as a basis for a learning algorithm.

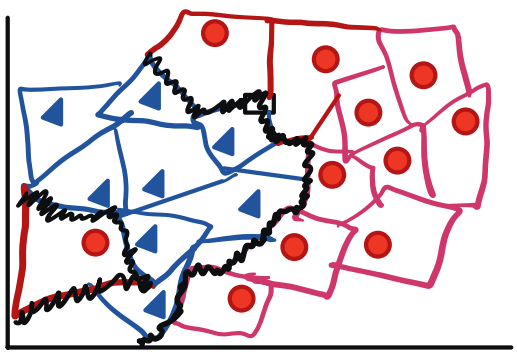
1-NN



Using this intuition → to classify a new point
- Find the most similar training example x'
- predict its class y'

- Voronoi tesselation
  - Partitions space into regions
  - boundary: Points at same distance from two different training examples
- Decision boundary
  - non linear, reflects classes well
  - compare to NB, DT, logistics
  - impressive for simple methods

K-NN



Algorithm is sensetive to outliers
- Single mislabeled dramatically changes boundary
- No confidence $P(Y|X)$
- use more than one NN to make decision (majority)

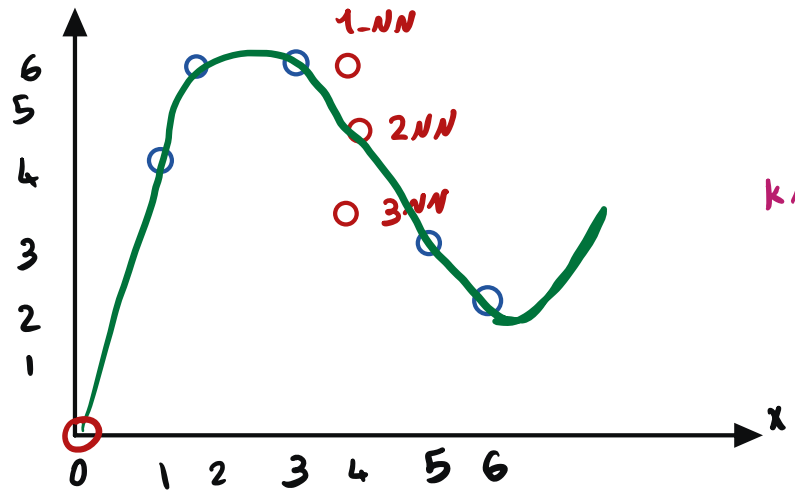
KNN classification algorithm

- compute distance $D(x, x_i)$ to every training example x_i
- select k closest instances $x_{i_1}, \dots, x_{i_k}$ and their tables $y_{i_1}, \dots, y_{i_k}$
- output the class y^* which is mast frequent in $y_{i_1}, \dots, y_{i_k}$

> Rating / No training

KNN _Regression

- compute distance $D(x, x_i)$ to every training example x_i
- select k closest instances $x_{i_1}, \dots, x_{i_k}$ and their tables $y_{i_1}, \dots, y_{i_k}$
- output the mean of $y_{i_1} \dots y_{i_k}$: $\hat{y} = F(x) = \frac{1}{k} \sum_{j=1}^{k} y_{r_j}$



KNN Regression in 1D

better @ interpolation

worse @ extrapolation

Choosing the value of k

k too large ⟶ most frequent class! CV, hyperparameter tuning

k too small ⟶ high variace ! tuning

Distance measures.

Euclidian $D(x,x') = \sqrt{\sum_d |x_d - x'_d|^2}$

not optimal but if you don't know anything about your domain it is your best choice

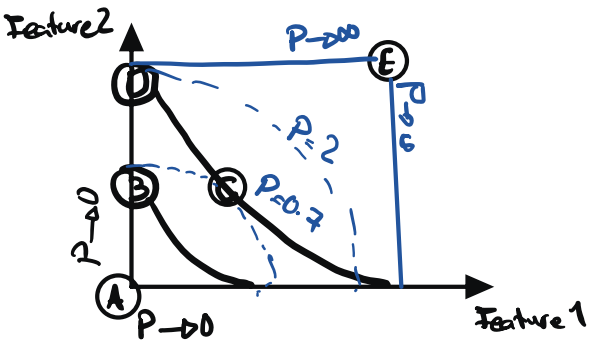
Hamming $D(x,x') = \sum_d 1_{x_d \neq x'_d}$

(categorical attributes) number of attributes where x, x' differ

Minkowski distance (P_norm)

$D(x,x') = \sqrt[P]{\sum_d |x_d - x'_d|^P}$

- $P=2$ Euclidian
- $P=1$ Manhattan
- $P \to \infty$ $|x_d - x'_d|$ logical "OR"
- $P=0$ logical "AND"



Kullback_Leibler (KL) divergence ⟶ For histograms / assymetric

$D(x,x') = -\sum_d x_d \log \frac{x_d}{x'_d}$

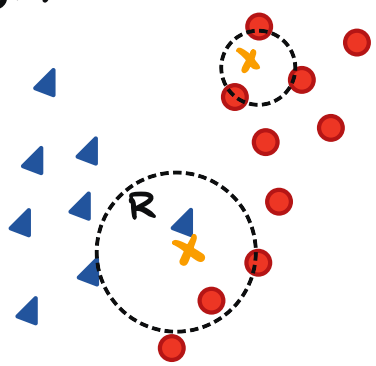
Custom Distance Function to your Farm

Practical issues:

- ties happen → equal number of +/- neighbours
  - → use odd k (doesn't solve multiclass)
  - → random_ Prior _ nearest
- missing values

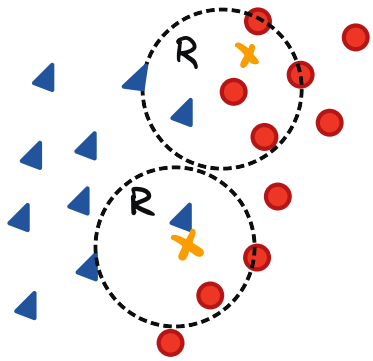
KNN and Parzen windows and kernels

3-NN



Constant number of neighbours

Parzen window



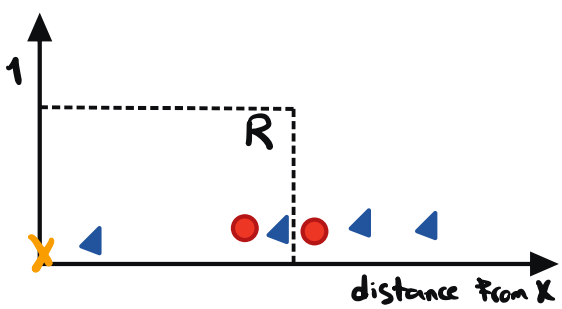
constant volume of space

$$f(x) = \text{Sgn}\left[\sum_{i: x_i \in R(x)} y_i\right]$$

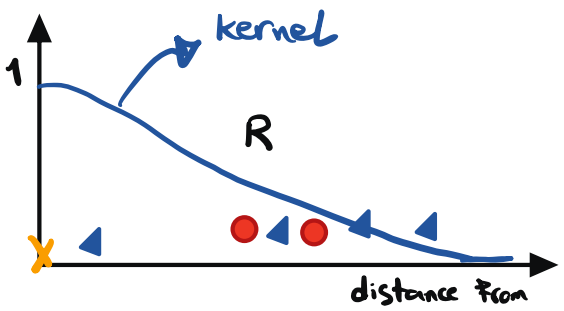
$$= \text{Sgn}\left[\sum_i y_i \cdot 1_{\|x_i - x\| \le R}\right]$$

rewrite based on a decay function

$$\text{Sgn}\left[\sum_i y_i k(x_i, x)\right]$$ VS



So with moving R it changes classification it is not desirable so what about replacing the ball function with smooth one so you weight more to the points based on how far away is from your x



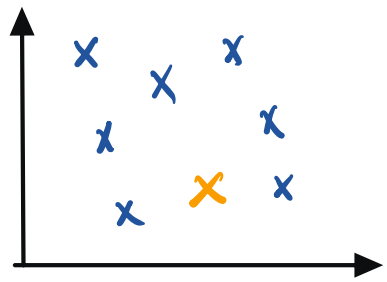
$$\text{Sgn}\left[\sum_i a_i y_i k(x_i, x)\right]$$

kernel SVM

Computational complexity of KNN

why KNN is slow?

what you see



Find nearest neighbours

what algorithm see

training set:

{(1,9), (2,3), (4,1), (3,7), (5,4),
...}

testing set:

(7,4)

Nearest neighbours → Compare one by one

Training O(d), but testing O(nd)

→ Reduce d ⇝ Feature selection/ Dimensionality Reduction

→ Reduce n ⇝ Do not compare to all training examples

- K-D trees
- inverted lists
- Locality-sensitive hashing